

Study of Queue, Stack and Linked List using GDB

Aim

To implement Queue, Stack, and Linked List in C and analyze their execution step-by-step using the GDB debugger.

Theory

Queue

A queue is a linear data structure that follows the FIFO (First In First Out) principle. Elements are inserted at the rear and removed from the front.

Stack

A stack follows the LIFO (Last In First Out) principle. Elements are inserted and removed from the top.

Linked List

A linked list is a dynamic data structure consisting of nodes, where each node contains data and a pointer to the next node.

GDB (GNU Debugger)

GDB is used to:

- Execute code step-by-step
- Track variable changes
- Understand control flow
- Debug errors efficiently

Experiment

Queue Implementation

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = value;
    printf("Enqueued %d | front = %d, rear = %d\n", value, front, rear);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Dequeued %d | new front = %d\n", queue[front], front + 1);
    front++;
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue (front -> rear): ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    enqueue(1);
    enqueue(2);
    display();
}
```

```

    dequeue();
    display();
    return 0;
}

```

```

bash - GDB Queue Final State & Memory

(gdb) info locals
No locals in current frame.
(gdb) print &queue
$7 = (int (*)[5]) 0x555555558040 <queue>
(gdb) x/5d 0x555555558040
0x555555558040 <queue>:  1  2  0  0  0
(gdb) print front
$8 = 1
(gdb) print rear
$9 = 1
Memory Address Analysis:
  queue[0] @ 0x555555558040 -> value: 1  (dequeued, front passed)
  queue[1] @ 0x555555558044 -> value: 2  (current front element)
  queue[2] @ 0x555555558048 -> value: 0  (empty)
✓ FIFO behaviour confirmed via memory inspection.
(gdb) quit
[Inferior 1 exited normally]

```

Linked List Implementation

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
    int data;
    struct Node* next;
};

```

```

struct Node* head = NULL;

```

```

void insert(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = head;
    head = newNode;
    printf("Inserted %d | head -> %d\n", value, head->data);
}

```

```

void display() {
    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

void freeList() {
    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

```

```

int main() {
    insert(10);
    insert(20);
    display();
    freeList();
    return 0;
}

```

Stack Implementation

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int stack[SIZE];
```

```
int top = -1;
```

```

void push(int value) {
    if (top == SIZE - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = value;
    printf("Pushed %d | top = %d\n", value, top);
}

```

```
}
```

```
void pop() {  
    if (top == -1) {  
        printf("Stack Underflow\n");  
        return;  
    }  
    printf("Popped %d | top = %d\n", stack[top], top - 1);  
    top--;  
}
```

```
void display() {  
    if (top == -1) {  
        printf("Stack is empty.\n");  
        return;  
    }  
    printf("Stack (top -> bottom): ");  
    for (int i = top; i >= 0; i--)  
        printf("%d ", stack[i]);  
    printf("\n");  
}
```

```
int main() {  
    push(10);  
    push(20);  
    display();  
    pop();  
    display();  
    return 0;  
}
```

```
bash - GDB Stack Debugging

user@lab:~$ gcc -g stack.c -o stack
user@lab:~$ gdb ./stack
GNU gdb (Ubuntu 12.1) 12.1 - Type 'help' for info.
Reading symbols from ./stack...
(gdb) break main
Breakpoint 1 at 0x11a9: file stack.c, line 22.
(gdb) run
Breakpoint 1, main () at stack.c:22
22      push(10);
(gdb) step
push (value=10) at stack.c:8
8      if (top == SIZE - 1) {
(gdb) next
10      stack[++top] = value;
(gdb) print top
$1 = 0
(gdb) print stack
$2 = {10, 0, 0, 0, 0}
(gdb) continue
Continuing.
Breakpoint 1, main () at stack.c:23
23      push(20);
(gdb) next
(gdb) print top
$3 = 1
(gdb) print stack
$4 = {10, 20, 0, 0, 0}
(gdb) next
24      pop();
(gdb) print top
$5 = 0 ← top decremented after pop()
✓ Stack push/pop verified. top updated correctly.
(gdb) quit
```

GDB Procedure

1. Compile with debug flag:

```
gcc -g program.c -o program
```

2. Start debugger:

```
gdb ./program
```

3. Set breakpoints:

break main

4. Run program:

run

5. Use commands:

- next → next line
- step → inside function
- print variable → check values

Result

The execution of Queue, Stack, and Linked List was successfully observed using GDB. Internal changes in variables and pointers were analyzed step-by-step.

Conclusion

GDB helps in understanding the internal working of data structures by allowing detailed inspection of execution flow, making debugging and learning more effective.